1. Introdução

Este documento apresenta um **plano de implementação prático**, focado em **o que fazer**, **que classes criar**, **que dados ler** e **como ligar tudo**, para o trabalho "Instalação Robótica" em Java. O objetivo é servir como guia direto para o desenvolvimento, sem incluir qualquer excerto de código.

---

2. Visão Geral do Funcionamento

O programa funciona em três grandes momentos:

1. **Leitura e validação dos dados iniciais** (mapa e entidades).
2. **Inicialização da simulação** (estruturas internas, estado inicial).
3. **Ciclo de comandos do utilizador**, onde a simulação avança por passos.

---

3. Classes Principais a Criar

3.1 Classe Main

Responsabilidade:

- Ponto de entrada do programa.
- Ler **apenas** os dados iniciais.
- Criar a simulação e arrancar o ciclo de comandos.

O que faz:

- Lê dimensões da instalação.
- Lê entidades até encontrar start.
- Valida erros fatais iniciais.
- Cria o objeto Simulation.

3.2 Classe Simulation

Responsabilidade:

- Controlar o estado global da simulação.

Atributos típicos:

- Step count
- Mapa da instalação
- Lista de robôs

- Lista de objetos
- Lista de tarefas

Métodos principais:

- Executar um step da simulação
- Pedir ações aos robôs
- Imprimir o estado atual

3.3 Classe Map / Installation

Responsabilidade:

- Representar a grelha da instalação.

Guarda:

- Largura e altura
- Estruturas para saber o que existe em cada célula

Funções principais:

- Verificar se uma célula está vazia
- Atualizar posições das entidades
- Imprimir o mapa exatamente no formato pedido

---

4. Entidades do Sistema

4.1 Classe abstrata Entity

Responsabilidade:

- Base comum para tudo o que existe no mapa.

Campos comuns:

- Coordenadas (x, y)

4.2 Obstáculo

- Nunca se move.
- Bloqueia o caminho dos robôs.

4.3 Objeto

- Tem ID próprio.
- Pode estar:
    - No mapa
    - A ser transportado
    - Associado a uma tarefa

4.4 Estação de Carregamento

- Associada a **um único robô**.
- Só esse robô a pode usar.

4.5 Robô

Campos importantes:

- ID
- Posição atual
- Carga
- Estado (IDLE, BUSY, CHARGING)
- Tarefa atual
- Estação de carregamento
- Caminho atual (lista de posições)

Responsabilidades:

- Avaliar tarefas
- Deslocar-se passo a passo
- Atualizar carga
- Pedir novo caminho quando muda de objetivo

---

5. Estados e Comportamento do Robô

Estados possíveis:

- **IDLE** – parado e disponível
- **BUSY** – a executar uma tarefa
- **CHARGING** – a restaurar carga

Comportamento por step:

- Se IDLE: tenta aceitar uma tarefa
- Se BUSY: anda uma célula
- Se CHARGING: aumenta a carga até 100%

---

6. Classe Task (Tarefa)

Guarda:

- Objeto associado
- Coordenadas de destino
- Estado da tarefa (livre, atribuída, concluída)
- Robô atribuído

Notas importantes:

- As tarefas são guardadas por ordem de criação.
- Um objeto só pode pertencer a uma tarefa.

---

7. Algoritmo de Caminhos (DFS)

Classe PathFinder / DFS

Responsabilidade:

- Calcular caminhos evitando obstáculos e entidades inválidas.

Usado para:

- Robô → Objeto
- Objeto → Destino
- Destino → Estação de carregamento

Informação devolvida:

- Lista de células do caminho
- Ou indicação de caminho impossível

---

8. Inputs a Ler

8.1 Dados Iniciais

Ordem exata:

1. Dimensões do mapa (W H)
2. Lista de entidades:
    - OBS x y
    - ROB x y cs-x cs-y
    - OBJ x y
3. Palavra startValidações imediatas:

- Dimensões mínimas e máximas
- Formato correto
- Coordenadas válidas
- Pelo menos 1 robô e 1 objeto

---

9. Comandos do Utilizador

Implementar uma classe ou gestor de comandos que:

- Lê uma linha
- Identifica o comando
- Executa a ação correspondente

Comandos:

- helpstepadd-task <obj-id> <x> <y>get-robot <id>exitTodos os comandos devem validar argumentos antes de alterar o estado.

---

10. Ciclo da Simulação (Step)

Em cada step:

1. Incrementar contador
2. Para cada robô:
    - Atualizar carga
    - Executar ação conforme estado
3. Atualizar mapa
4. Imprimir:
    - Step count
    - Mapa
    - Lista de robôs
    - Lista de tarefas

---

11. Ordem Recomendada de Implementação

1. Leitura e validação do input inicial
2. Representação do mapa
3. Entidades básicas
4. Impressão correta do estado
5. Robôs parados (IDLE)
6. Caminhos DFS
7. Execução de tarefas
8. Sistema de carga
9. Comandos
10. Ajustes finos e testes Mooshak

---

12. Conclusão

Este plano descreve **exatamente o que implementar**, **em que ordem** e **que classes usar**, servindo como guia prático para desenvolver o trabalho com segurança, organização e alinhamento total com o enunciado.